Complete Analysis of GenAI Code

Name of the Code Files

sudoku.cpp (ChatGPT-generated)

sudoku_solver.cpp(Claude-generated)

sudoku.cpp (corrected/commented version, based on the Claude-generated code)

puzzle1.txt

puzzle2.txt

puzzle3.txt

puzzle4.txt

puzzle5.txt

How They Were Accessed

Claude and ChatGPT, both accessed via their web browser interfaces (claude.ai and chatgpt.com).

Prompt:

Write a recursive object-oriented program to find the solution to the five puzzle files.

The output for each solution should include:

- The puzzle file name (e.g., puzzle1.txt)
- The puzzle from the file printed out
- The solution to the puzzle printed out, or if no solution is found, “No solution found” printed out

The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.

generate C++ code using objects, not functions, to implement the following requirements.

Code Generated: Claude

```
// Sudoku solver: recursive depth-first search with backtracking.
// Finds and prints ALL solutions for each puzzle file.
//
// Build:  g++ -std=c++17 -O2 -o sudoku sudoku.cpp
// Run:    ./sudoku                    (solves puzzle1.txt .. puzzle5.txt)
//         ./sudoku file1.txt file2.txt  (solves the files given)
```

```
#include <array>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

// ----------------------------------------------------------------------------
// Board: holds a 9x9 grid and knows how to read, print, and validate itself.
// ----------------------------------------------------------------------------
class Board {
public:
    static const int SIZE = 9;
    static const int BLOCK = 3;

    Board() {
        for (auto &row : grid) row.fill(0);
    }

    // Reads a puzzle from a file. Digits 1-9 are givens; '_' is blank.
    // Whitespace is ignored. Returns false if the file can't be read
    // or doesn't contain exactly 81 cells.
    bool load(const std::string &filename) {
        std::ifstream in(filename);
        if (!in) return false;

        int count = 0;
        char ch;
        while (in.get(ch) && count < SIZE * SIZE) {
            if (ch >= '1' && ch <= '9') {
                grid[count / SIZE][count % SIZE] = ch - '0';
                ++count;
            } else if (ch == '_' || ch == '0' || ch == '.') {
                grid[count / SIZE][count % SIZE] = 0;
                ++count;
            }
            // anything else (spaces, newlines) is skipped
        }
        return count == SIZE * SIZE;
    }

    int get(int r, int c) const { return grid[r][c]; }
    void set(int r, int c, int v) { grid[r][c] = v; }
    bool isEmpty(int r, int c) const { return grid[r][c] == 0; }

    // Can digit v legally be placed at (r, c) given the current grid?
    bool canPlace(int r, int c, int v) const {
        for (int i = 0; i < SIZE; ++i) {
            if (grid[r][i] == v) return false;  // row
```

```
        if (grid[i][c] == v) return false;  // column
    }
    int br = (r / BLOCK) * BLOCK;
    int bc = (c / BLOCK) * BLOCK;
    for (int i = 0; i < BLOCK; ++i)
        for (int j = 0; j < BLOCK; ++j)
            if (grid[br + i][bc + j] == v) return false;  // block
    return true;
}

// Candidate digits for cell (r, c).
std::vector<int> candidates(int r, int c) const {
    std::vector<int> result;
    for (int v = 1; v <= SIZE; ++v)
        if (canPlace(r, c, v)) result.push_back(v);
    return result;
}

// Finds the empty cell with the fewest candidates (speeds up the search
// without changing which solutions are found). Returns false if the
// board is full.
bool findBestEmptyCell(int &outR, int &outC) const {
    size_t best = SIZE + 1;
    bool found = false;
    for (int r = 0; r < SIZE; ++r) {
        for (int c = 0; c < SIZE; ++c) {
            if (!isEmpty(r, c)) continue;
            size_t n = candidates(r, c).size();
            if (n < best) {
                best = n;
                outR = r;
                outC = c;
                found = true;
                if (best == 0) return true;  // dead end, stop early
            }
        }
    }
    return found;
}

// Checks that the givens don't already violate a rule.
bool givensAreConsistent() const {
    Board copy = *this;
    for (int r = 0; r < SIZE; ++r) {
        for (int c = 0; c < SIZE; ++c) {
            int v = copy.grid[r][c];
            if (v == 0) continue;
            copy.grid[r][c] = 0;
```

```
            bool ok = copy.canPlace(r, c, v);
            copy.grid[r][c] = v;
            if (!ok) return false;
        }
    }
    return true;
}

void print(std::ostream &out) const {
    for (int r = 0; r < SIZE; ++r) {
        if (r > 0 && r % BLOCK == 0) out << "------+-------+------\n";
        for (int c = 0; c < SIZE; ++c) {
            if (c > 0 && c % BLOCK == 0) out << "| ";
            out << (grid[r][c] == 0 ? '_' : static_cast<char>('0' + grid[r][c]));
            if (c < SIZE - 1) out << ' ';
        }
        out << '\n';
    }
}

private:
    std::array<std::array<int, SIZE>, SIZE> grid;
};

// ----------------------------------------------------------------------------
// SudokuSolver: recursive DFS with backtracking. Collects every solution.
// ----------------------------------------------------------------------------
class SudokuSolver {
public:
    explicit SudokuSolver(const Board &puzzle) : working(puzzle) {}

    // Runs the search and returns all solutions found (possibly none).
    const std::vector<Board> &solveAll() {
        solutions.clear();
        if (working.givensAreConsistent()) search();
        return solutions;
    }

private:
    Board working;
    std::vector<Board> solutions;

    // Recursive step: fill one empty cell, recurse, then undo (backtrack).
    void search() {
        int r = 0, c = 0;
        if (!working.findBestEmptyCell(r, c)) {
            // No empty cells left: this is a complete, valid solution.
            solutions.push_back(working);
```

```
            return;
        }

        std::vector<int> options = working.candidates(r, c);
        // If options is empty we fall straight through: dead end, backtrack.
        for (int v : options) {
            working.set(r, c, v);
            search();
            working.set(r, c, 0);  // backtrack
        }
    }
};

// ----------------------------------------------------------------------------
// PuzzleRunner: loads one puzzle file, solves it, and reports the results.
// ----------------------------------------------------------------------------
class PuzzleRunner {
public:
    explicit PuzzleRunner(const std::string &file) : filename(file) {}

    void run(std::ostream &out) const {
        out << "==============================\n";
        out << "Puzzle file: " << filename << "\n\n";

        Board puzzle;
        if (!puzzle.load(filename)) {
            out << "Error: could not read a valid 9x9 puzzle from this file.\n\n";
            return;
        }

        out << "Puzzle:\n";
        puzzle.print(out);
        out << '\n';

        SudokuSolver solver(puzzle);
        const std::vector<Board> &solutions = solver.solveAll();

        if (solutions.empty()) {
            out << "No solution found\n\n";
            return;
        }

        out << solutions.size()
            << (solutions.size() == 1 ? " solution found:\n\n" : " solutions found:\n\n");
        for (size_t i = 0; i < solutions.size(); ++i) {
            if (solutions.size() > 1) out << "Solution " << (i + 1) << ":\n";
            else out << "Solution:\n";
            solutions[i].print(out);
```

```
            out << '\n';
        }
    }

private:
    std::string filename;
};

// ----------------------------------------------------------------------------
// Application: decides which files to process and runs them.
// ----------------------------------------------------------------------------
class Application {
public:
    Application(int argc, char *argv[]) {
        if (argc > 1) {
            for (int i = 1; i < argc; ++i) files.push_back(argv[i]);
        } else {
            for (int i = 1; i <= 5; ++i)
                files.push_back("puzzle" + std::to_string(i) + ".txt");
        }
    }

    int run() const {
        for (const std::string &f : files) PuzzleRunner(f).run(std::cout);
        return 0;
    }

private:
    std::vector<std::string> files;
};

int main(int argc, char *argv[]) {
    return Application(argc, argv).run();
}
```

Code Generated: ChatGPT

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class Sudoku
{
private:
```

```
    char grid[9][9];
    string fileName;
    int solutionCount;

public:
    // Constructor
    Sudoku(string file)
    {
        fileName = file;
        solutionCount = 0;
    }

    // Read the puzzle from a file
    bool readPuzzle()
    {
        ifstream inputFile(fileName);

        if (!inputFile)
        {
            cout << "Could not open " << fileName << endl;
            return false;
        }

        for (int row = 0; row < 9; row++)
        {
            for (int col = 0; col < 9; col++)
            {
                inputFile >> grid[row][col];
            }
        }

        inputFile.close();
        return true;
    }

    // Print the Sudoku grid
    void printGrid()
    {
        for (int row = 0; row < 9; row++)
        {
            for (int col = 0; col < 9; col++)
            {
                cout << grid[row][col] << " ";

                if ((col + 1) % 3 == 0 && col != 8)
                    cout << "| ";
            }
```

```
        cout << endl;

        if ((row + 1) % 3 == 0 && row != 8)
            cout << "------+-------+------" << endl;
    }
}

// Check whether a digit can be placed in a cell
bool isValid(int row, int col, char digit)
{
    // Check the row
    for (int i = 0; i < 9; i++)
    {
        if (grid[row][i] == digit)
            return false;
    }

    // Check the column
    for (int i = 0; i < 9; i++)
    {
        if (grid[i][col] == digit)
            return false;
    }

    // Find the top-left corner of the 3x3 block
    int startRow = (row / 3) * 3;
    int startCol = (col / 3) * 3;

    // Check the 3x3 block
    for (int r = startRow; r < startRow + 3; r++)
    {
        for (int c = startCol; c < startCol + 3; c++)
        {
            if (grid[r][c] == digit)
                return false;
        }
    }

    return true;
}

// Recursive backtracking solver
void solve()
{
    // Find the next empty cell
    int row = -1;
    int col = -1;
```

```
    for (int r = 0; r < 9; r++)
    {
        for (int c = 0; c < 9; c++)
        {
            if (grid[r][c] == '_')
            {
                row = r;
                col = c;
                break;
            }
        }

        if (row != -1)
            break;
    }

    // No empty cells means a complete solution was found
    if (row == -1)
    {
        solutionCount++;

        cout << "\nSolution " << solutionCount << ":" << endl;
        printGrid();
        return;
    }

    // Try every possible digit
    for (char digit = '1'; digit <= '9'; digit++)
    {
        if (isValid(row, col, digit))
        {
            // Place the digit
            grid[row][col] = digit;

            // Recursively solve the remaining puzzle
            solve();

            // Backtrack to try other possible solutions
            grid[row][col] = '_';
        }
    }
}

// Display the puzzle and all its solutions
void run()
{
    cout << "\n================================" << endl;
    cout << "File: " << fileName << endl;
```

```
        cout << "==================================" << endl;

        if (!readPuzzle())
            return;

        cout << "\nOriginal Puzzle:" << endl;
        printGrid();

        cout << "\nSearching for solutions..." << endl;

        solve();

        if (solutionCount == 0)
        {
            cout << "\nNo solution found." << endl;
        }
        else
        {
            cout << "\nTotal solutions found: "
                 << solutionCount << endl;
        }
    }
};

int main()
{
    Sudoku puzzle1("puzzle1.txt");
    Sudoku puzzle2("puzzle2.txt");
    Sudoku puzzle3("puzzle3.txt");
    Sudoku puzzle4("puzzle4.txt");
    Sudoku puzzle5("puzzle5.txt");

    puzzle1.run();
    puzzle2.run();
    puzzle3.run();
    puzzle4.run();
    puzzle5.run();

    return 0;
}
```

Code Analysis:

Correctness: Both programs correctly use recursion and backtracking to solve sudoku puzzles, but the Claude code is better because it reads the files character by character and

handles blank spaces (like _ and .). The ChatGPT has strict and fragile formatting. The Claude code also checks if the puzzle is valid before solving, which prevents errors.

Execution Time: Both share the same theoretical speed limits, however, the Claude code is faster in practice because it uses a strategy called Minimum Remaining Values heuristic to pick out the best next empty cell to fill first. This cuts down searching and avoids dead ends faster compared to the ChatGPT's code.

Time Complexity: Both programs have the same worst-case time complexity of $O(9^N)$, where N is the number of empty cells. In the absolute worst-case scenario, both have to test every single possibility. Even so, the second version performs much better on average in real-world tests because of its smart searching strategy.

Space Complexity: The first program uses slightly less memory because it prints each solution right away. The second program stores all solutions in a list before printing them all at the end. Both use a very small amount of memory for the recursion stack ($O(81)$), but the second version takes up more memory if a puzzle has a huge number of solutions.

Maintainability: Despite both codes not having a lot of comments, the second program is much easier to maintain because the code is divided into separate parts like Board, SudokuSolver, PuzzleRunner and Application, which is more efficient and allows to debug much easier compared to ChatGPT code running it under one class. The Claude code also allows you to input files, rather than hard coding them, which allows you to swap the files and write your own puzzles.

Which Code Is Better?

The Claude code is the stronger implementation overall. Execution time is significantly faster in the Claude version due to the Minimum Remaining Values (MRV) heuristic pruning the search tree, even though both share the same theoretical O(9^N) worst-case time complexity. Space complexity is slightly more favorable in the ChatGPT version since it prints solutions immediately rather than storing all of them in a vector, though both maintain a minimal recursion stack. However, what the Claude code does much better is in its correctness and maintainability; it features robust character-by-character file parsing that handles multiple blank characters, validates starting givens, and cleanly separates responsibilities into modular classes with flexible command-line support. Overall, the Claude version is the better starting point because its superior execution speed, robust error handling, and clean object-oriented architecture far outweigh its slightly higher memory usage for multi-solution puzzles.

How the code was improved:

Error & Edge Case Handling

- A puzzle file that is missing, is a folder, or can't be opened is skipped with a specific error message that names the file, and the program carries on with the next file instead of crashing.

- Malformed puzzle files (empty file, fewer or more than 81 cells, or an invalid character) are rejected with a message that says what is wrong, including the line number for invalid characters and extra data, so a bad file never enters the solver.
- Puzzles whose given digits already break a rule (for example, two 5s in one row) print “No solution found” with a reason that names the row and column, instead of being searched needlessly.
- A puzzle that is already completely solved is accepted and printed as its single solution.
- An empty or nearly empty grid, which has an astronomical number of solutions, stops after 100 solutions with a note that more may exist, so the program can’t run forever or flood the output.
- Windows line endings (`\r\n`), trailing spaces, and a UTF-8 byte-order mark are handled, so files saved on Windows read correctly.
- Reading and solving are each wrapped in `try`/`catch` with a custom `PuzzleException`, so one bad file can’t stop the others, and unexpected errors (such as out-of-memory) are reported instead of crashing.
- The program returns exit code 1 if any file could not be read (with a summary of how many failed), 0 if all files were read, and 2 for a fatal error, which is reported on the error stream.

Maintainability

- Every line of code has a comment, and each class (`PuzzleException`, `Board`, `SudokuSolver`, `PuzzleRunner`, `Application`) has a header comment explaining its role.
- The file header now documents how to build, how to run, and what each exit code means.
- The solution limit is a single named constant (`MAX_SOLUTIONS`), and the accepted blank markers (`_`, `0`, `.`) are defined in one place, so both are easy to change.
- The loader reads into a temporary grid and only commits it once the whole file is valid, so a failed load never leaves a half-filled board.

Updated code:

```
// Sudoku solver: recursive depth-first search with backtracking (MRV cell ordering).  // File header: what the program does
// Finds and prints ALL solutions (up to a safety limit) for each puzzle file.  // File header: output behaviour
//
// Build:  g++ -std=c++17 -O2 -Wall -o sudoku sudoku.cpp  // How to compile (C++17 is needed for <filesystem>)
// Run:    ./sudoku                    (solves puzzle1.txt .. puzzle5.txt)  // Default usage
//         ./sudoku file1.txt file2.txt  (solves the files given)  // Usage with explicit file names
// Exit code: 0 if every file was read successfully, 1 if any file had an error.  // Exit status meaning

#include <array>       // std::array: fixed-size rows of the grid
```

```
#include <cctype>      // std::isspace, std::isprint: character classification
#include <filesystem>  // std::filesystem: check that a file exists and is a regular file
#include <fstream>     // std::ifstream: reading puzzle files
#include <iostream>    // std::cout, std::ostream: printing results
#include <stdexcept>   // std::runtime_error, std::exception: error handling
#include <string>      // std::string: file names and messages
#include <system_error>// std::error_code: non-throwing filesystem checks
#include <vector>      // std::vector: candidate lists and solution lists

// ---------------------------------------------------------------------------  // Section divider
// PuzzleException: the error type thrown when a puzzle file cannot be used.  // Describes the class purpose
// ---------------------------------------------------------------------------  // Section divider
class PuzzleException : public std::runtime_error {  // Custom exception derived from runtime_error
public:  // Public members follow
   explicit PuzzleException(const std::string &message)  // Constructor takes the error message
       : std::runtime_error(message) {}  // Pass the message to the base class
};  // End of PuzzleException

// ---------------------------------------------------------------------------  // Section divider
// Board: holds a 9x9 grid and knows how to read, print, and validate itself.  // Describes the class purpose
// ---------------------------------------------------------------------------  // Section divider
class Board {  // Start of the Board class
public:  // Public members follow
   static const int SIZE = 9;   // Number of rows and columns in the grid
   static const int BLOCK = 3;  // Width and height of one sub-grid (block)

   Board() {  // Default constructor
      for (auto &row : grid) row.fill(0);  // Start with every cell empty (0 means empty)
   }  // End of constructor

   // Reads a puzzle from a file. Digits 1-9 are givens; '_' is blank.  // Describes load()
   // Throws PuzzleException with a specific message if anything is wrong.  // Describes the error behaviour
   void load(const std::string &filename) {  // Load a puzzle from the named file
      namespace fs = std::filesystem;  // Short alias for the filesystem namespace
      std::error_code ec;  // Receives filesystem errors without throwing

      if (filename.empty())  // Edge case: an empty file name was supplied
         throw PuzzleException("no file name was given");  // Report the problem
      if (!fs::exists(filename, ec) || ec)  // Edge case: file is missing (or the check itself failed)
         throw PuzzleException("file does not exist");  // Report the problem
      if (!fs::is_regular_file(filename, ec) || ec)  // Edge case: path is a directory or special file
         throw PuzzleException("path is not a regular file");  // Report the problem
```

```
    std::ifstream in(filename, std::ios::binary);  // Open in binary mode so we see raw bytes (\r, BOM)
    if (!in)  // Edge case: file exists but cannot be opened (e.g. permissions)
        throw PuzzleException("file could not be opened (check permissions)");  // Report the problem

    char bom[3] = {0, 0, 0};  // Buffer for a possible UTF-8 byte-order mark
    in.read(bom, 3);  // Try to read the first three bytes
    bool hasBom = in.gcount() == 3 &&  // A BOM needs exactly three bytes...
            static_cast<unsigned char>(bom[0]) == 0xEF &&  // ...first byte 0xEF
            static_cast<unsigned char>(bom[1]) == 0xBB &&  // ...second byte 0xBB
            static_cast<unsigned char>(bom[2]) == 0xBF;  // ...third byte 0xBF
    if (!hasBom) {  // If there was no BOM, those bytes are real content
        in.clear();  // Clear the EOF flag in case the file was shorter than 3 bytes
        in.seekg(0);  // Go back to the start so nothing is lost
    }  // End of BOM handling

    std::array<std::array<int, SIZE>, SIZE> temp;  // Read into a temporary grid so errors leave this Board untouched
    for (auto &row : temp) row.fill(0);  // Initialise the temporary grid to empty
    int count = 0;  // How many cells have been read so far
    int line = 1;   // Current line number, used in error messages
    char ch;  // The character currently being examined

    while (in.get(ch)) {  // Read the file one character at a time
        if (ch == '\n') {  // A newline moves to the next line
            ++line;  // Track the line number
            continue;  // Nothing else to do for a newline
        }  // End of newline handling
        if (std::isspace(static_cast<unsigned char>(ch)))  // Spaces, tabs and '\r' (Windows) are ignored
            continue;  // Skip whitespace

        bool isDigit = (ch >= '1' && ch <= '9');  // Is this a given digit 1-9?
        bool isBlank = (ch == '_' || ch == '0' || ch == '.');  // Accept '_' (required) plus '0' and '.' as blanks

        if (!isDigit && !isBlank) {  // Edge case: any other character is invalid
            std::string shown = std::isprint(static_cast<unsigned char>(ch))  // Show printable characters directly
                                ? std::string("'") + ch + "'"  // Printable: wrap in quotes
                                : std::string("a non-printable character");  // Otherwise describe it generically
            throw PuzzleException("invalid character " + shown + " on line " + std::to_string(line));  // Report the problem
        }  // End of invalid-character check

        if (count >= SIZE * SIZE)  // Edge case: more than 81 cells in the file
```

```
            throw PuzzleException("more than 81 cells found (extra data on line " +
std::to_string(line) + ")");  // Report the problem

            temp[count / SIZE][count % SIZE] = isDigit ? (ch - '0') : 0;  // Store the digit, or 0 for a
blank
            ++count;  // One more cell has been read
        }  // End of read loop

        if (in.bad())  // Edge case: a low-level I/O error happened mid-read
            throw PuzzleException("I/O error while reading the file");  // Report the problem
        if (count == 0)  // Edge case: the file is empty or only whitespace
            throw PuzzleException("file is empty");  // Report the problem
        if (count < SIZE * SIZE)  // Edge case: not enough cells
            throw PuzzleException("only " + std::to_string(count) + " cells found; expected 81");
// Report the problem

        grid = temp;  // Everything was valid, so commit the new grid
    }  // End of load()

    int get(int r, int c) const { return grid[r][c]; }  // Read the digit at row r, column c (0 = empty)
    void set(int r, int c, int v) { grid[r][c] = v; }  // Write digit v at row r, column c
    bool isEmpty(int r, int c) const { return grid[r][c] == 0; }  // True when the cell has no digit

    // Can digit v legally be placed at (r, c) given the current grid?  // Describes canPlace()
    bool canPlace(int r, int c, int v) const {  // Check row, column and block for v
        for (int i = 0; i < SIZE; ++i) {  // Walk along the row and column together
            if (grid[r][i] == v) return false;  // v already appears in this row
            if (grid[i][c] == v) return false;  // v already appears in this column
        }  // End of row/column loop
        int br = (r / BLOCK) * BLOCK;  // Top row of the block containing (r, c)
        int bc = (c / BLOCK) * BLOCK;  // Left column of the block containing (r, c)
        for (int i = 0; i < BLOCK; ++i)  // Loop over the block's rows
            for (int j = 0; j < BLOCK; ++j)  // Loop over the block's columns
                if (grid[br + i][bc + j] == v) return false;  // v already appears in this block
        return true;  // No conflicts, so v can be placed
    }  // End of canPlace()

    // Candidate digits for cell (r, c).  // Describes candidates()
    std::vector<int> candidates(int r, int c) const {  // Collect every legal digit for the cell
        std::vector<int> result;  // Holds the legal digits
        for (int v = 1; v <= SIZE; ++v)  // Try each digit 1-9
            if (canPlace(r, c, v)) result.push_back(v);  // Keep it if it does not conflict
        return result;  // Return the list (may be empty = dead end)
    }  // End of candidates()

    // Finds the empty cell with the fewest candidates (MRV heuristic).  // Describes
findBestEmptyCell()
    // Returns false if the board is full.  // Describes the return value
```

```
    bool findBestEmptyCell(int &outR, int &outC) const {  // Output the chosen row and column
        size_t best = SIZE + 1;  // Fewest candidates seen so far (start above the maximum of 9)
        bool found = false;  // Becomes true once any empty cell is seen
        for (int r = 0; r < SIZE; ++r) {  // Scan every row
            for (int c = 0; c < SIZE; ++c) {  // Scan every column
                if (!isEmpty(r, c)) continue;  // Skip cells that already have a digit
                size_t n = candidates(r, c).size();  // How many digits could go here
                if (n < best) {  // Is this the most constrained cell so far?
                    best = n;  // Remember its candidate count
                    outR = r;  // Remember its row
                    outC = c;  // Remember its column
                    found = true;  // At least one empty cell exists
                    if (best == 0) return true;  // Zero candidates is a dead end; stop looking
                }  // End of better-cell check
            }  // End of column loop
        }  // End of row loop
        return found;  // False means the board has no empty cells
    }  // End of findBestEmptyCell()

    // Describes the first rule violation among the given digits, or "" if none.  // Describes firstConflict()
    std::string firstConflict() const {  // Used to detect contradictory puzzles up front
        Board copy = *this;  // Work on a copy so this Board is not modified
        for (int r = 0; r < SIZE; ++r) {  // Check every row
            for (int c = 0; c < SIZE; ++c) {  // Check every column
                int v = copy.grid[r][c];  // The digit in this cell
                if (v == 0) continue;  // Empty cells cannot conflict
                copy.grid[r][c] = 0;  // Temporarily remove the digit
                bool ok = copy.canPlace(r, c, v);  // Could it be placed again without a clash?
                copy.grid[r][c] = v;  // Put the digit back
                if (!ok)  // A clash means two equal digits share a row, column or block
                    return "the given " + std::to_string(v) + " at row " + std::to_string(r + 1) +  // Build a human-readable message
                           ", column " + std::to_string(c + 1) +  // Add the column (1-based)
                           " conflicts with another " + std::to_string(v) +  // Say which digit clashes
                           " in its row, column or block";  // Say where the clash is
            }  // End of column loop
        }  // End of row loop
        return "";  // No conflicts found
    }  // End of firstConflict()

    void print(std::ostream &out) const {  // Print the grid with block separators
        for (int r = 0; r < SIZE; ++r) {  // One output line per row
            if (r > 0 && r % BLOCK == 0) out << "------+-------+------\n";  // Horizontal divider between blocks
            for (int c = 0; c < SIZE; ++c) {  // Print each cell in the row
```

```
            if (c > 0 && c % BLOCK == 0) out << "| ";  // Vertical divider between blocks
            out << (grid[r][c] == 0 ? '_' : static_cast<char>('0' + grid[r][c]));  // '_' for empty, else
the digit
            if (c < SIZE - 1) out << ' ';  // Space between cells (not after the last)
         }  // End of column loop
         out << '\n';  // End the row
      }  // End of row loop
   }  // End of print()

private:  // Private data follows
   std::array<std::array<int, SIZE>, SIZE> grid;  // The 9x9 grid of digits (0 = empty)
};  // End of Board

// ---------------------------------------------------------------------------  // Section divider
// SudokuSolver: recursive DFS with backtracking. Collects solutions.  // Describes the class
purpose
// ---------------------------------------------------------------------------  // Section divider
class SudokuSolver {  // Start of the SudokuSolver class
public:  // Public members follow
   // Safety limit: a nearly empty grid has astronomically many solutions,  // Explains why the
limit exists
   // so we stop collecting after this many instead of running forever.  // Explains what the
limit does
   static const size_t MAX_SOLUTIONS = 100;  // Change this value to allow more or fewer
solutions

   explicit SudokuSolver(const Board &puzzle) : working(puzzle) {}  // Copy the puzzle into
the working board

   // Runs the search and returns the solutions found (possibly none).  // Describes solveAll()
   const std::vector<Board> &solveAll() {  // Entry point for solving
      solutions.clear();  // Discard results from any earlier run
      limitReached = false;  // Reset the limit flag
      conflict = working.firstConflict();  // Look for contradictory givens before searching
      if (conflict.empty()) search();  // Only search when the givens are consistent
      return solutions;  // Hand back the solutions
   }  // End of solveAll()

   bool hitLimit() const { return limitReached; }  // True if the search stopped at
MAX_SOLUTIONS
   const std::string &getConflict() const { return conflict; }  // Message about contradictory
givens ("" if none)

private:  // Private data follows
   Board working;  // The board being modified during the search
   std::vector<Board> solutions;  // Every complete solution found
   bool limitReached = false;  // Set when MAX_SOLUTIONS is hit
   std::string conflict;  // Describes contradictory givens, if any
```

```
    // Recursive step: fill one empty cell, recurse, then undo (backtrack).  // Describes search()
    void search() {  // The recursive depth-first search
        if (limitReached) return;  // Stop immediately once the limit is hit

        int r = 0, c = 0;  // Will hold the chosen cell
        if (!working.findBestEmptyCell(r, c)) {  // No empty cells left?
            solutions.push_back(working);  // Then the grid is a complete solution; save it
            if (solutions.size() >= MAX_SOLUTIONS) limitReached = true;  // Stop if we have
collected enough
            return;  // Back up to try other possibilities
        }  // End of base case

        std::vector<int> options = working.candidates(r, c);  // Legal digits for the chosen cell
        for (int v : options) {  // Try each candidate (an empty list means a dead end)
            working.set(r, c, v);  // Place the candidate
            search();  // Recurse to fill the remaining cells
            working.set(r, c, 0);  // Backtrack: remove the candidate
            if (limitReached) return;  // Unwind quickly if the limit was hit deeper down
        }  // End of candidate loop
    }  // End of search()
};  // End of SudokuSolver

// ----------------------------------------------------------------------  // Section divider
// PuzzleRunner: loads one puzzle file, solves it, and reports the results.  // Describes the
class purpose
// ----------------------------------------------------------------------  // Section divider
class PuzzleRunner {  // Start of the PuzzleRunner class
public:  // Public members follow
    explicit PuzzleRunner(const std::string &file) : filename(file) {}  // Remember which file to
process

    // Returns true if the file was read and processed, false on a read error.  // Describes run()
    bool run(std::ostream &out) const {  // Process the file and print to 'out'
        out << "==============================\n";  // Visual separator between puzzles
        out << "Puzzle file: " << filename << "\n\n";  // Always show the file name first

        Board puzzle;  // Will hold the puzzle read from the file
        try {  // Catch any problem so one bad file cannot crash the program
            puzzle.load(filename);  // Read and validate the file
        } catch (const PuzzleException &e) {  // Expected, descriptive file errors
            out << "ERROR: " << e.what() << "\n\n";  // Show the specific reason
            return false;  // Tell the caller this file failed
        } catch (const std::exception &e) {  // Anything unexpected (e.g. out of memory)
            out << "ERROR: unexpected problem while reading: " << e.what() << "\n\n";  // Show
what we know
            return false;  // Tell the caller this file failed
        }  // End of try/catch
```

```
        out << "Puzzle:\n";  // Heading for the original puzzle
        puzzle.print(out);  // Print the puzzle as read from the file
        out << '\n';  // Blank line for readability

        try {  // Guard the solving step as well
            SudokuSolver solver(puzzle);  // Create a solver for this puzzle
            const std::vector<Board> &solutions = solver.solveAll();  // Find the solutions

            if (solutions.empty()) {  // No solution exists
                out << "No solution found\n";  // Required message from the assignment
                if (!solver.getConflict().empty())  // If the givens were contradictory...
                    out << "Reason: " << solver.getConflict() << "\n";  // ...explain why
                out << '\n';  // Blank line for readability
                return true;  // The file was read fine; it just has no solution
            }  // End of no-solution case

            out << solutions.size()  // Report how many solutions were found
                << (solutions.size() == 1 ? " solution found:\n\n" : " solutions found:\n\n");  //
Singular or plural wording
            for (size_t i = 0; i < solutions.size(); ++i) {  // Print each solution
                if (solutions.size() > 1) out << "Solution " << (i + 1) << ":\n";  // Number them when
there are several
                else out << "Solution:\n";  // Plain heading for a single solution
                solutions[i].print(out);  // Print the solved grid
                out << '\n';  // Blank line after each grid
            }  // End of solution loop

            if (solver.hitLimit())  // Warn if the output was cut off by the safety limit
                out << "Note: stopped after " << SudokuSolver::MAX_SOLUTIONS  // Mention the
limit value
                    << " solutions (the puzzle may have more).\n\n";  // Say there may be more
        } catch (const std::exception &e) {  // Unexpected failure while solving
            out << "ERROR: unexpected problem while solving: " << e.what() << "\n\n";  // Show
what we know
            return false;  // Tell the caller this file failed
        }  // End of try/catch
        return true;  // File processed successfully
    }  // End of run()

private:  // Private data follows
    std::string filename;  // The puzzle file this runner handles
};  // End of PuzzleRunner

// ----------------------------------------------------------------------  // Section divider
// Application: decides which files to process and runs them.  // Describes the class purpose
// ----------------------------------------------------------------------  // Section divider
class Application {  // Start of the Application class
```

```
public:  // Public members follow
    Application(int argc, char *argv[]) {  // Build the list of files from the command line
        if (argc > 1) {  // The user passed file names
            for (int i = 1; i < argc; ++i) files.push_back(argv[i]);  // Use each argument as a file
name
        } else {  // No arguments given
            for (int i = 1; i <= 5; ++i)  // Default to puzzle1.txt .. puzzle5.txt
                files.push_back("puzzle" + std::to_string(i) + ".txt");  // Build each default file name
        }  // End of argument handling
    }  // End of constructor

    int run() const {  // Process every file and return the exit code
        int failures = 0;  // Count of files that could not be read
        for (const std::string &f : files)  // Go through each file in order
            if (!PuzzleRunner(f).run(std::cout)) ++failures;  // Run it and count any failure
        if (failures > 0)  // Summarise problems at the end
            std::cout << failures << " file(s) could not be processed.\n";  // Tell the user how many
failed
        return failures > 0 ? 1 : 0;  // Non-zero exit code signals an error to the shell
    }  // End of run()

private:  // Private data follows
    std::vector<std::string> files;  // The list of puzzle files to process
};  // End of Application

int main(int argc, char *argv[]) {  // Program entry point
    try {  // Last line of defence against uncaught exceptions
        return Application(argc, argv).run();  // Create the application and run it
    } catch (const std::exception &e) {  // Any exception that escaped everything else
        std::cerr << "Fatal error: " << e.what() << '\n';  // Report it on the error stream
        return 2;  // Exit code 2 means a fatal error
    }  // End of try/catch
}  // End of main
```